Bayesian Structural Time Series

ECON 8310: Business Forecasting — Lecture 11, Part 1

Department of Economics

University of Nebraska at Omaha

August 26, 2026

Lecture Outline

1. Why Bayesian Time Series
2. The Structural Decomposition
3. Fitting It in PyMC
4. Reading the Forecast
5. Key Takeaways

Why Bayesian Time Series

Not a more accurate forecast — a different deliverable.

What Every Model So Far Gave You

Ten weeks of forecasting, and almost every model returned the same object: **one number per period**. ETS, ARIMA, random forests, XGBoost, LSTMs — a point forecast, and at best an interval derived from asymptotic theory that nobody checked.

For a planner that is often not enough. *How much stock do I need so the chance of a stockout is under 5%?* is a question about a distribution, and a point forecast cannot answer it however accurate it is.

> A Bayesian time series model returns a **full predictive distribution for every future period**, plus a distribution over each component — the trend, the seasonality, and the noise, each with its own uncertainty.

Set expectations now. This will not beat XGBoost on RMSE, and on our data it barely beats the seasonal-naive benchmark. The argument for it is what it hands you, not where it ranks — and we will hold it to that standard at the end of this lecture.

Three Things That Become Possible

Priors carry knowledge	"Weekly growth is very unlikely to exceed 20%" is a sentence you can put in the model, rather than hoping the data implies it.
Components get error bars	Not just a forecast, but "how confident am I that there is a trend at all?" — separately from the seasonality.
Uncertainty compounds honestly	Forecast 52 weeks out and the interval widens as it should, because the model propagates uncertainty forward instead of assuming it away.

That third row is the one people underestimate. A prediction interval from ARIMA widens too, but according to a formula derived under assumptions you did not check. Here the widening is a consequence of the model you actually wrote down.

And the price: it is slower, it needs priors you have to defend, and — as we will see — it can fail silently in ways a least-squares fit cannot.

The Structural Decomposition

Write the series as a sum of parts you can name, and put a prior on each.

Structural Time Series: One Equation

The whole idea is additive, and you have seen the shape before — it is the GAM decomposition from Lecture 3, with priors attached.

$$y_t = \underbrace{\mu_t}_{\text{level}} + \underbrace{s_t}_{\text{seasonality}} + \underbrace{\varepsilon_t}_{\text{noise}}, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$$

Each piece is a thing a business person can name, and each gets its own posterior. "Demand is growing" is a statement about μ_t; "December is our peak" is a statement about s_t; "how noisy is this store?" is σ.

> This is why the approach is called *structural*. You are not fitting a black box to a curve — you are stating what you believe the series is made of, and asking the data how much of each part there is.

Compare ARIMA, where trend and seasonality are handled by differencing and the parameters are correlations rather than quantities anyone can interpret.

The Level: A Random Walk With a Prior On It

The simplest useful trend is a **local level** — the series has a level, and the level itself drifts:

$$\mu_t = \mu_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, \sigma^2_{\text{level}})$$

Read it as a sentence: this week's level is last week's level, plus a shock. And σ_{level} — which you put a prior on — controls **how fast the level is allowed to move**.

Small σ_{level}	A nearly constant level. Rigid, but stable far into the future.
Large σ_{level}	A level that chases recent data. Fits well, and forecasts wander.

That is a familiar trade in new clothing — it is the smoothing parameter from Lecture 1's exponential smoothing, and the penalty strength from Lecture 6, now expressed as a prior you can argue about.

Seasonality as Fourier Terms

Fifty-two weekly seasonal parameters would be a lot to estimate. Instead, build seasonality out of a few smooth waves:

$$s_t = \sum_{k=1}^{K} \left[a_k \sin\left(\tfrac{2\pi k t}{P}\right) + b_k \cos\left(\tfrac{2\pi k t}{P}\right) \right]$$

P is the period — for our weekly data, $P = 52$. K is how many waves you allow: $K = 1$ is a single smooth annual cycle, larger K lets the shape get more detailed. Each a_k, b_k is just a coefficient with a Normal prior.

> **You have met this exact device twice.** It is how Prophet encodes seasonality in Lecture 3, and it is the Fourier feature block we engineered by hand for Homework 4. The difference now is that a_k and b_k come back as *distributions*.

K trades flexibility against overfitting, exactly like spline knots. We use $K = 3$ — six coefficients standing in for fifty-two.

Fitting It in PyMC

The model in code, and the check you run before believing any of it.

The Model in PyMC

Local level + Fourier seasonality

```
with pm.Model() as model:

sigma_level = pm.Exponential("sigma_level", 10.0)

z           = pm.Normal("z", 0, 1, shape=n)   # non-centred

level = pm.Deterministic("level", pm.math.cumsum(z * sigma_level))

beta = pm.Normal("beta", 0, 0.5, shape=2*K)      # Fourier

sigma = pm.Exponential("sigma", 1.0)

pm.Normal("obs", mu=level + pm.math.dot(X, beta),

sigma=sigma, observed=y_scaled)

idata = pm.sample(1500, tune=2000, chains=4, target_accept=0.95)
```

Line for line this is the equations above — `cumsum` of scaled Normal draws *is* a random walk. **Standardize y first**: these priors assume unit scale, and on raw units of 20,000 they would be absurdly tight.

The First Fit Failed — Here Is What That Looks Like

Written the obvious way, with `pm.GaussianRandomWalk`, this model samples badly — even at `target_accept=0.95`. Lecture 10's checks caught it:

	First attempt	After the fix	Want
$\hat{R}$	1.022	**1.002**	< 1.01
ESS (bulk)	252	**2,376**	> 400
Divergences	0	0	0

Nothing crashed. The model returned a forecast, and the forecast looked reasonable. Only the diagnostics said the chains had not agreed with each other.

The fix was reparameterization, not more samples. Writing the walk as `cumsum(z * sigma)` with $z \sim \mathcal{N}(0, 1)$ — the *non-centred* form — gives the sampler a geometry it can navigate. BMCP §4.6.1 covers exactly this, and you will meet it again next week.

Reading the Forecast

The point estimate, the interval, and whether either is any use.

Does It Work? The Point Forecast

CA_1 FOODS, 52 weeks held out, forecast from the fitted model.

Model	Test RMSE
Bayesian structural TS	**1,587**
Seasonal naive (same week last year)	1,660

It beats the benchmark by about 4%. That is a real improvement and a small one, and after nine weeks of this course you should be suspicious of anyone who reports it as a triumph.

It is also worth remembering what this model was given: the series and a calendar. No lag features, no price, no SNAP flags, no store identity. The XGBoost model in Homework 4 had 46 engineered predictors and thirty series to learn from.

So the point forecast is not the reason to do this. The next slide is.

The Interval Is the Product

Every forecast comes with a 94% predictive interval. Two questions about it, and they are different questions.

	Measured	**Reading**
Calibrated?	100% of weeks fell inside	Not overconfident — but it should be about 94%, so it is conservative
Useful?	Width $\approx$ 11,700 units, on a mean of 19,700	About $\pm$30%. Honest, and too wide to order against

Passing the first does not excuse failing the second. A model saying "somewhere between 14,000 and 26,000" is telling the truth and helping nobody. The cause is structural — a level free to walk for 52 steps drifts, and the model says so. The fix is a model with a stronger opinion: a damped level, or covariates that pin it down.

But notice what just happened. We could *ask* whether the uncertainty was honest. For every point-forecast model in this course, that question had no answer.

Key Takeaways

What structural models are for.

Lecture 11 Part 1: Key Takeaways

1. A **structural** model writes the series as named parts — level, seasonality, noise — and gives each one a prior and a posterior.
2. The **local level** is a random walk whose step size σ_{level} you put a prior on. Small means rigid, large means it chases the data — the same trade as a smoothing parameter or a penalty.
3. **Fourier terms** encode seasonality in a few coefficients instead of 52. Same device as Prophet in Lecture 3 and the features you built in Homework 4.
4. **Check the sampler before the forecast.** Our first fit had $\hat{R} = 1.02$ and ESS $= 252$ while returning a plausible-looking answer. Reparameterizing fixed it.
5. The point forecast beat seasonal naive by **4%**. That is not the argument.
6. The interval was **calibrated but too wide to act on** — and being able to establish that is the actual product.

Next: hierarchical models — what to do when you have thirty series and some of them barely have data.

References I